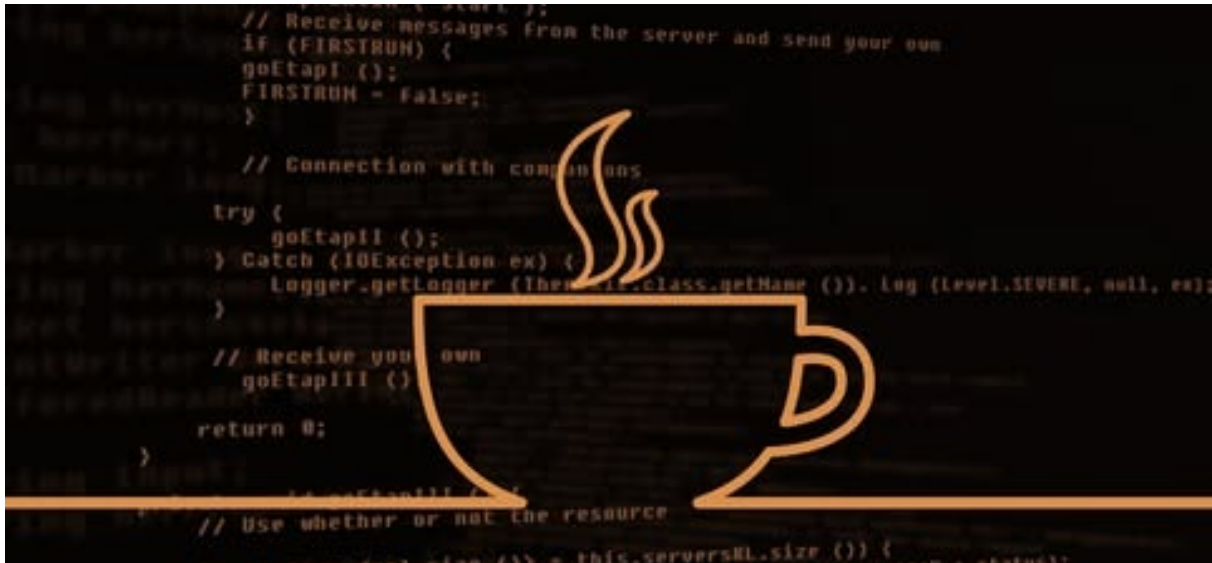




TP4 - Java

🕒 Date de création	1 octobre 2026 08:15
🏷️ Étiquettes	

1. Arbres

Question 1

La structure de l'arbre est importante pour, par exemple, représenter des hiérarchies, des décisions ou bien encore du découpage spatial.

Pour démontrer :

- Situations techniques
 - Système de fichiers
 - Auto-complétion
- Situations fonctionnelles
 - Organisation des entreprises
 - Prise de décisions
 - Catalogue de produits

Question 2 / 5 / 6 / 7

```

package Arbres;

public class SinglyListArbreClass<T> {
    private T value;
    private SinglyListArbreClass<T> left;
    private SinglyListArbreClass<T> right;

    public SinglyListArbreClass() {
        this.value = value;
        this.left = null;
        this.right = null;
    }

    public SinglyListArbreClass(T value, SinglyListArbreClass<T> left, SinglyListArbreClass<T> right) {
        this.value = value;
        this.left = left;
        this.right = right;
    }

    public T getValue() {
        return value;
    }

    public SinglyListArbreClass<T> getLeft() {
        return left;
    }

    public SinglyListArbreClass<T> getRight() {
        return right;
    }

    public void setLeft(SinglyListArbreClass<T> left) {
        this.left = left;
    }

    public void setRight(SinglyListArbreClass<T> right) {
        this.right = right;
    }
}

```

```

public String parcoursPrefixe() {
    StringBuilder sb = new StringBuilder();
    sb.append(this.value).append(" ");
    if (this.left != null) {
        sb.append(this.left.parcoursPrefixe());
    }
    if (this.right != null) {
        sb.append(this.right.parcoursPrefixe());
    }
    return sb.toString();
}

public String toString() {
    return parcoursPrefixe();
}

public static void main (String[] args) {
    SinglyListArbreClass<String> deux = new SinglyListArbreClass<>("2");
    SinglyListArbreClass<String> huit = new SinglyListArbreClass<>("8");
    SinglyListArbreClass<String> vingt = new SinglyListArbreClass<>("20");
    SinglyListArbreClass<String> cinq = new SinglyListArbreClass<>("5");

    SinglyListArbreClass<String> Plus = new SinglyListArbreClass<>("+", cinq, deux);
    SinglyListArbreClass<String> Fois = new SinglyListArbreClass<>("*", Plus, huit);
    SinglyListArbreClass<String> dix = new SinglyListArbreClass<>("10", cinq, vingt);

    System.out.println("Parcours préfixé : " + dix.parcoursPrefixe());
    System.out.println("Parcours préfixé de ((5 + 2) * 8) :

```

```
" + Foix); }  
}
```

Question 3 : Façons différentes de parcourir :

- Parcours en largeur
- Parcours en profondeur
 - **Préfixé** (*Racine / Gauche / Droite*)
 - **Infixé** (*Gauche / Racine / Droite*)
 - **Postfixé** (*Gauche / Droite / Racine*)

Question 4 : Quelle est le nom de la méthode de parcours qui commence par la racine ?

→ Parcours en profondeur Préfixe

Question 8

```
package Arbres;  
  
import java.util.List;  
import java.util.ArrayList;  
  
public class ArbreNAire<T> {  
    private T value;  
    private List<ArbreNAire<T>> children;  
  
    public ArbreNAire(T value) {  
        this.value = value;  
        this.children = new ArrayList<>();  
    }  
  
    public T getValue() {  
        return value;  
    }  
  
    public List<ArbreNAire<T>> getChildren() {  
        return children;  
    }  
}
```

```

}

public ArbreNAire<T> ajoutChild(ArbreNAire<T> child){
    this.children.add(child);
}

public String parcoursPrefixe() {
    StringBuilder sb = new StringBuilder();
    sb.append(this.value).append(" ");
    for (ArbreNAire<T> child : this.children) {
        sb.append(child.parcoursPrefixe());
    }
    return sb.toString();
}

public String toString() {
    return parcoursPrefixe();
}

public static void main (String[] args) {
    ArbreNAire<String> cinq = new ArbreNAire<>("5");
    ArbreNAire<String> six = new ArbreNAire<>("6");
    ArbreNAire<String> deux = new ArbreNAire<>("2");

    deux.ajoutChild(cinq);
    deux.ajoutChild(six);

    ArbreNAire<String> trois = new ArbreNAire<>("3");
    ArbreNAire<String> quatre = new ArbreNAire<>("4");
    ArbreNAire<String> un = new ArbreNAire<>("1");

    un.ajoutChild(deux);
    un.ajoutChild(trois);
    un.ajoutChild(quatre);

    System.out.println("Parcours préfixé N-aire : " + un.parcoursPrefixe());
}

```

```
}  
}
```

Question 9

```
package Arbres;  
public class Arbre {  
    public static void main(String[] args) {  
        ArbreNAire<String> html = new ArbreNAire<>("html");  
        ArbreNAire<String> finHtml = new ArbreNAire<>("/html");  
  
        ArbreNAire<String> head = new ArbreNAire<>("head");  
        ArbreNAire<String> finHead = new ArbreNAire<>("/head");  
  
        ArbreNAire<String> body = new ArbreNAire<>("body");  
        ArbreNAire<String> finBody = new ArbreNAire<>("/body");  
  
        ArbreNAire<String> title = new ArbreNAire<>("title");  
        ArbreNAire<String> finTitle = new ArbreNAire<>("/title");  
  
        ArbreNAire<String> h1 = new ArbreNAire<>("h1");  
        ArbreNAire<String> finH1 = new ArbreNAire<>("/h1");  
  
        ArbreNAire<String> p = new ArbreNAire<>("p");  
        ArbreNAire<String> finP = new ArbreNAire<>("/p");  
  
        h1.ajoutChild(new ArbreNAire<>("Titre niveau 1"));  
        h1.ajoutChild(finH1);  
  
        p.ajoutChild(new ArbreNAire<>("Ceci est un paragraphe"));  
        p.ajoutChild(finP);  
  
        head.ajoutChild(title);
```

```

        title.ajoutChild(new ArbreNAire<>("Page test"));
        title.ajoutChild(finTitle);

        body.ajoutChild(h1);
        body.ajoutChild(p);

        html.ajoutChild(head);
        html.ajoutChild(finHead);
        html.ajoutChild(body);
        html.ajoutChild(finBody);
        html.ajoutChild(finHtml);

        System.out.println("Parcours du DOM HTML : ");
        System.out.println(html);
    }
}

```

2. Application des arbres

Question 1

- **Pour JSON & Arbre de jeu d'échecs** : ArbreNAire <T> est parfaitement adaptée car elle gère une liste d'enfants (List <ArbreNAire<T>>) et un type générique T.
- **Pour Arbre de décision & Expression arithmétique** : SinglyListArbreClass<T> convient car ces arbres sont généralement binaires.

Résultat avec Postfixe : 2 3 + 5 * 9 1 3 4 + + 3 * - -

On ajoute la fonction parcoursPostfixe() dans la classe SinglyListArbreClass

```

public String parcoursPostfixe() {
    StringBuilder sb = new StringBuilder();
    if (this.left != null) {
        sb.append(this.left.parcoursPostfixe()).append("
");
    }
    if (this.right != null) {

```

```

        sb.append(this.right.parcoursPostfixe()).append("
");
    }
    sb.append(this.value);
    return sb.toString();
}

```

3. Piles

Question 1

```

package Piles;
import java.util.EmptyStackException;

public class Pile<T> {
    private static class Node<T> {
        private T value;
        private Node<T> next;

        public Node(T value, Node<T> next) {
            this.value = value;
            this.next = next;
        }
    }

    private Node<T> top = null;
    private int size = 0;

    public T empiler(T value) {
        top = new Node<>(value, top);
        size++;
        return value;
    }

    public T depiler() {
        if (isEmpty())
        {
            throw new EmptyStackException();
        }
    }
}

```

```

    }
    T value = top.value;
    top = top.next;
    size--;
    return value;
}

public T first() {
    if (isEmpty())
    {
        throw new EmptyStackException();
    }
    return top.value;
}

public boolean isEmpty() {
    return top == null;
}

public int size() {
    return size;
}
}

```

Question 3

```

package Piles;

public class NPI {
    public static int evaluator(String expression) {
        Pile<Integer> pile = new Pile<>();
        for (int i = 0; i < expression.length(); i++) {
            char c = expression.charAt(i);
            if (Character.isDigit(c)) {
                pile.empiler(Character.getNumericValue(c));
            }
            else if (c == '+' || c == '-' || c == '*') {
                int y = pile.depiler();
            }
        }
    }
}

```

```

        int x = pile.depiler();
        int r = 0;

        switch (c) {
            case '+':
                r = x + y;
                break;
            case '-':
                r = x - y;
                break;
            case '*':
                r = x * y;
                break;
        }

        pile.empiler(r);
    }
}
return pile.depiler();
}

public static void main(String[] args) {
    String expr1 = "2 3 + 5 * 9 1 3 4 + + 3 * - -";
    System.out.println("Résultat de [" + expr1 + "] = " +
evaluer(expr1));

    String expr2 = "52+83-*";
    System.out.println("Résultat de [" + expr2 + "] = " + e
valuer(expr2));
}
}

```